LAPORAN PRAKTIKUM

ALGORITMA PEMROGRAMAN 2

MODUL 10 – PENCARIAN NILAI MAX/MIN

NAMA : AMANDA SEPTIANA SALSABILA

NIM : 109082500145

KELAS : S1IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Konsep Pencarian dalam Pemrograman
   Pencarian (searching) merupakan operasi fundamental dalam ilmu komputer yang bertujuan untuk menemukan elemen tertentu dalam sekumpulan data. Operasi ini menjadi dasar bagi berbagai aplikasi, mulai dari pengelolaan basis data, analisis statistik, hingga sistem pengambilan keputusan. Dalam konteks algoritma, pencarian nilai ekstrim (maksimum/minimum) adalah bentuk khusus dari operasi pencarian yang berfokus pada identifikasi nilai terbesar atau terkecil dalam suatu himpunan data.

2. Definisi Nilai Ekstrim
   Nilai ekstrim merujuk pada nilai terbesar (maksimum) atau terkecil (minimum) yang terdapat dalam sekumpulan data. Penentuan nilai ekstrim memiliki peran penting dalam :
   - Analisis statistik deskriptif
   - Pengambilan keputusan berbasis data
   - Optimasi sistem dan proses
   - Evaluasi performa atau kriteria tertentu

3. Prinsip Kerja Algoritma Pencarian Max/Min
   Algoritma pencarian nilai ekstrim bekerja dengan pendekatan sekuensial, dengan prinsip sebagai berikut :
   - Inisialisasi : Menetapkan elemen pertama sebagai kandidat nilai ekstrim sementara.
   - Iterasi Komparatif : Melakukan perbandingan berurutan antara kandidat ekstrim dengan elemen-elemen berikutnya.
   - Pembaruan Kondisional : Memperbarui kandidat ekstrim apabila ditemukan nilai yang lebih ekstrem sesuai kriteria (lebih besar untuk maksimum, lebih kecil untuk minimum).
   - Terminasi : Setelah seluruh elemen telah diproses, nilai kandidat yang tersisa merupakan nilai ekstrim yang valid.

4. Kompleksitas Algoritma
   Algoritma pencarian nilai ekstrim memiliki karakteristik kompleksitas sebagai berikut :

- Kompleksitas Waktu : O(n), dengan n merupakan jumlah elemen data, karena setiap elemen harus diakses dan dibandingkan tepat satu kali.
- Kompleksitas Ruang : O(1), karena algoritma hanya memerlukan variabel tambahan untuk menyimpan kandidat nilai atau indeks ekstrim, tanpa bergantung pada ukuran input.

5. Representasi Hasil Pencarian
   Hasil dari operasi pencarian nilai ekstrim dapat direpresentasikan dalam dua bentuk :
   - Nilai Ekstrim : Mengembalikan nilai maksimum atau minimum itu sendiri.
   - Indeks Ekstrim : Mengembalikan posisi atau indeks dari nilai ekstrim dalam struktur data.

   Pemilihan bentuk representasi disesuaikan dengan kebutuhan aplikasi. Pengembalian indeks umumnya lebih fleksibel karena memungkinkan akses ke seluruh atribut data yang terkait, terutama pada tipe data terstruktur.

6. Penerapan pada Tipe Data
   - Tipe Data Dasar
     Pada array dengan tipe data primitif (integer, float, dll), pencarian nilai ekstrim dilakukan langsung terhadap nilai elemen. Operasi ini relatif sederhana dan efisien.
   - Tipe Data Terstruktur
     Pada array yang berisi elemen bertipe struktur (struct atau record), pencarian dilakukan berdasarkan field tertentu. Pendekatan ini memungkinkan identifikasi entitas lengkap yang memenuhi kriteria ekstrim, bukan hanya nilai field-nya saja.

7. Pertimbangan Implementasi
   Beberapa aspek teknis yang perlu diperhatikan dalam mengimplementasikan algoritma pencarian nilai ekstrim :
   - Penanganan kasus batas, seperti array kosong atau berisi satu elemen.
   - Pemilihan strategi inisialisasi kandidat ekstrim yang tepat.
   - Konsistensi dalam penggunaan indeks, terutama terkait konvensi pengindeksan pada bahasa pemrograman tertentu.
   - Efisiensi dalam akses memori dan operasi perbandingan.

8. Relevansi dalam Pengembangan Perangkat Lunak
   Pemahaman terhadap algoritma pencarian nilai ekstrim merupakan kompetensi dasar dalam pengembangan perangkat lunak. Algoritma ini tidak hanya digunakan secara mandiri, tetapi juga menjadi komponen penyusun dalam algoritma yang lebih kompleks, seperti pengurutan (sorting), seleksi, dan optimasi. Penguasaan konsep ini mendukung kemampuan analitis dan pemecahan masalah dalam konteks pemrograman struktural maupun berorientasi objek.

B. Guided

1. Implementasi Pencarian Nilai Minimum dan Indeksnya pada Array Bertipe Data Dasar Menggunakan Bahasa Pemrograman Go

```
package main

import "fmt"

type arrInt [2023]int

func terkecil_1(tabInt arrInt, n int) int {
    var min int = tabInt[0]
    var j int = 1

    for j < n {
        if min > tabInt[j] {
            min = tabInt[j]
        }
        j = j + 1
    }
    return min
}

func terkecil_2(tabInt arrInt, n int) int {
    var idx int = 0
    var j int = 1

    for j < n {
        if tabInt[idx] > tabInt[j] { // [9, 8, 5, 6]
            idx = j
        }
        j = j + 1
    }
    return idx
}

func main() {
```

```
    var n int
    var arr arrInt

    fmt.Print("Masukkan jumlah data: ")
    fmt.Scan(&n)

    fmt.Println("Masukkan elemen array :")

    for i := 0; i < n; i++ {
        fmt.Scan(&arr[i])
    }

    min := terkecil_1(arr, n)
    idx := terkecil_2(arr, n)

    fmt.Println("Nilai terkecil: ", min)
    fmt.Println("Indeks nilai terkecil: ", idx)
}
```

Output :

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   Code

PS C:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septiana-Salsabila> go run "c:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septi
ana-Salsabila\Week 9 - Modul 10\Guided\Guided-1\1.go"
Masukkan jumlah data: 5
Masukkan elemen array :
9
3
7
1
5
Nilai terkecil:  1
Indeks nilai terkecil:  3
PS C:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septiana-Salsabila>
```

2. Implementasi Pencarian Nilai IPK Maksimum dan Minimum pada Array Bertipe Data Terstruktur Menggunakan Bahasa Pemrograman Go

```
package main

import "fmt"

type mahasiswa struct {
    nama string
    nim  string
    IPK  float64
}

type arrMahasiswa [3]mahasiswa

//data index 0 = nama, nim, IPK
//data index 1 = nama, nim, ipk
//data index 2 = nama, nim, ipk

func IPKTerbesar(array []mahasiswa) (mahasiswa, int) {
```

```
    var mhsTerbesar mahasiswa = array[0] //nilai awal
    var idxDitemukan int = 0
    var i int

    for i = 1; i < len(array); i++ {
        if array[i].IPK > mhsTerbesar.IPK {
            mhsTerbesar = array[i]
            idxDitemukan = i
        }
    }

    return mhsTerbesar, idxDitemukan
}

func IPKTerkecil(array []mahasiswa) (mahasiswa, int) {
    var mhsTerkecil mahasiswa = array[0] //nilai awal
    var idxDitemukan int = 0
    var i int

    for i = 1; i < len(array); i++ {
        if array[i].IPK < mhsTerkecil.IPK {
            mhsTerkecil = array[i]
            idxDitemukan = i
        }
    }

    return mhsTerkecil, idxDitemukan
}

func main() {
    var arrMhs arrMahasiswa
    var i int

    for i = 0; i < len(arrMhs); i++ {
        fmt.Println("INPUT DATA MAHASISWA INDEX KE-", i)
        fmt.Print("Masukkan nama : ")
        fmt.Scan(&arrMhs[i].nama)
        fmt.Print("Masukkkan nim : ")
        fmt.Scan(&arrMhs[i].nim)
        fmt.Print("Masukkan IPK : ")
        fmt.Scan(&arrMhs[i].IPK)
        fmt.Println("==================================")
    }
    fmt.Println()

    var mhsMaks, mhsMin mahasiswa
    var idxMaks, idxMin int

    mhsMaks, idxMaks = IPKTerbesar(arrMhs[:])

    mhsMin, idxMin = IPKTerkecil(arrMhs[:])
```

```
    fmt.Println("DATA IPK TERKECIL DAN TERBESAR")
    fmt.Println("=== IPK TERBESAR ===")
    fmt.Println("IPK Terbesar = ", mhsMaks.IPK)
    fmt.Println("Atas nama = ", mhsMaks.nama)
    fmt.Println("nim = ", mhsMaks.nim)
    fmt.Println("ditemukan pada indeks ke-", idxMaks)
    fmt.Println()
    fmt.Println("=== IPK TERKECIL ===")
    fmt.Println("IPK Terkecil = ", mhsMin.IPK)
    fmt.Println("Atas nama = ", mhsMin.nama)
    fmt.Println("nim = ", mhsMin.nim)
    fmt.Println("ditemukan pada indeks ke-", idxMin)
}
```

Output :

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   Code

PS C:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septiana-Salsabila> go run "c:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septi
ana-Salsabila\Week 9 - Modul 10\Guided\Guided-2\2.go"
INPUT DATA MAHASISWA INDEX KE- 0
Masukkan nama : Amanda
Masukkkan nim : 109082500145
Masukkan IPK : 3.88
==================================
INPUT DATA MAHASISWA INDEX KE- 1
Masukkan nama : Sarah
Masukkkan nim : 109082530023
Masukkan IPK : 3.90
==================================
INPUT DATA MAHASISWA INDEX KE- 2
Masukkan nama : Liana
Masukkkan nim : 109082500166
Masukkan IPK : 3.89
==================================

DATA IPK TERKECIL DAN TERBESAR
=== IPK TERBESAR ===
IPK Terbesar =  3.9
Atas nama =  Sarah
nim =  109082530023
ditemukan pada indeks ke- 1

=== IPK TERKECIL ===
IPK Terkecil =  3.88
Atas nama =  Amanda
nim =  109082500145
ditemukan pada indeks ke- 0
PS C:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septiana-Salsabila>
```

C. Unguided

1. Soal Unguided 1

   Sebuah program digunakan untuk mendata berat anak kelinci yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat anak kelinci yang akan dijual.

   Masukan terdiri dari sekumpulan bilangan, yang mana bilangan pertama adalah bilangan bulat N yang menyatakan banyaknya anak kelinci yang akan

ditimbang beratnya. Selanjutnya N bilangan riil berikutnya adalah berat dari anak kelinci yang akan dijual.

Keluaran terdiri dari dua buah bilangan riil yang menyatakan berat kelinci terkecil dan terbesar.

Jawaban :

```
package main

import "fmt"

type arrReal [1000]float64

func terkecil(tabReal arrReal, n int) float64 {
    var min float64 = tabReal[0]
    var j int = 1
    for j < n {
        if min > tabReal[j] {
            min = tabReal[j]
        }
        j = j + 1
    }
    return min
}

func terbesar(tabReal arrReal, n int) float64 {
    var max float64 = tabReal[0]
    var j int = 1
    for j < n {
        if max < tabReal[j] {
            max = tabReal[j]
        }
        j = j + 1
    }
    return max
}

func main() {
    var n int
    var berat arrReal

    fmt.Scan(&n)
    for i := 0; i < n; i++ {
        fmt.Scan(&berat[i])
    }

    min := terkecil(berat, n)
    max := terbesar(berat, n)
```

```
    fmt.Printf("%.2f %.2f\n", min, max)
}
```

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                    Code

PS C:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septiana-Salsabila> go run "c:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septi
ana-Salsabila\Week 9 - Modul 10\Unguided\unguided-no1\no1.go"
5
2.5 3.8 1.9 4.2 2.1
1.90 4.20
PS C:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septiana-Salsabila> go run "c:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septi
ana-Salsabila\Week 9 - Modul 10\Unguided\unguided-no1\no1.go"
6
1.2 3.5 4.8 2.9 5.1 3.3
1.20 5.10
PS C:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septiana-Salsabila> go run "c:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septi
ana-Salsabila\Week 9 - Modul 10\Unguided\unguided-no1\no1.go"
4
2.3 1.8 3.1 6.7
1.80 6.70
PS C:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septiana-Salsabila>
```

Penjelasan :

Program diatas dibuat menggunakan bahasa pemrograman Golang untuk mengimplementasikan algoritma pencarian nilai ekstrim guna menentukan berat anak kelinci terkecil dan terbesar dari sekumpulan data yang diinputkan. Program ini mendeklarasikan tipe data array berkapasitas 1000 elemen bertipe float64 sebagai wadah penyimpanan data, kemudian memisahkan logika pencariannya ke dalam dua fungsi independen, yaitu terkecil dan terbesar. Kedua fungsi tersebut bekerja secara sekuensial dengan menetapkan elemen pertama sebagai nilai acuan awal, lalu membandingkannya satu per satu dengan elemen berikutnya melalui perulangan yang berjalan selama indeks masih kurang dari jumlah total data. Apabila selama iterasi ditemukan nilai yang lebih kecil atau lebih besar dari acuan, variabel penyimpan akan langsung diperbarui hingga seluruh elemen selesai diverifikasi. Pada fungsi utama, program menangani pembacaan jumlah data dan elemen-elemennya melalui input standar, memanggil kedua fungsi pencarian secara berurutan, serta mencetak hasilnya dengan format dua angka desimal, sehingga alur eksekusi berjalan runtut dan mampu menghasilkan keluaran berupa dua bilangan riil yang mewakili nilai minimum dan maksimum.

2. Soal Unguided 2

   Sebuah program digunakan untuk menentukan tarif ikan yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat ikan yang akan dijual.

Masukan terdiri dari dua baris, yang mana baris pertama terdiri dari dua bilangan bulat x dan y. Bilangan x menyatakan banyaknya ikan yang akan dijual, sedangkan y adalah banyaknya ikan yang akan dimasukan ke dalam wadah. Baris kedua terdiri dari sejumlah x bilangan riil yang menyatakan banyaknya ikan yang akan dijual.

Keluaran terdiri dari dua baris. Baris pertama adalah kumpulan bilangan riil yang menyatakan total berat ikan disetiap wadah (jumlah wadah tergantung pada nilai x dan y, urutan ikan yang dimasukan ke dalam wadah sesuai urutan pada masukan baris ke-2). Baris kedua adalah sebuah bilangan riil yang menyatakan berat rata-rata ikan di setiap wadah.

Jawaban :

```
package main

import "fmt"

type arrReal [1000]float64

func hitungTotalWadah(arr arrReal, x, y int) (arrReal, int)
{
    var totalWadah arrReal
    var i, j, idx int
    idx = 0
    for i = 0; i < x; i = i + y {
        var total float64 = 0
        for j = 0; j < y; j++ {
            if i+j < x {
                total = total + arr[i+j]
            }
        }
        totalWadah[idx] = total
        idx = idx + 1
    }
    return totalWadah, idx
}

func hitungRataRata(totalWadah arrReal, numWadah int)
float64 {
    var sum float64 = 0
    var i int
    for i = 0; i < numWadah; i++ {
        sum = sum + totalWadah[i]
    }
    return sum / float64(numWadah)
}
```

```go
func main() {
    var x, y int
    var berat arrReal

    fmt.Scan(&x, &y)
    for i := 0; i < x; i++ {
        fmt.Scan(&berat[i])
    }

    totalWadah, numWadah := hitungTotalWadah(berat, x, y)

    for i := 0; i < numWadah; i++ {
        fmt.Printf("%.2f", totalWadah[i])
        if i < numWadah-1 {
            fmt.Print(" ")
        }
    }
    fmt.Println()

    rataRata := hitungRataRata(totalWadah, numWadah)
    fmt.Printf("%.2f\n", rataRata)
}
```

Output :

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   Code

PS C:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septiana-Salsabila> go run "c:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septi
ana-Salsabila\Week 9 - Modul 10\Unguided\unguided-no2\no2.go"
6 2
1.5 2.0 1.8 2.2 1.9 2.1
3.50 4.00 4.00
3.83
PS C:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septiana-Salsabila> go run "c:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septi
ana-Salsabila\Week 9 - Modul 10\Unguided\unguided-no2\no2.go"
3 2
0.123 0.456 0.789
0.58 0.79
0.68
PS C:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septiana-Salsabila> go run "c:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septi
ana-Salsabila\Week 9 - Modul 10\Unguided\unguided-no2\no2.go"
5 1
10.5 20.3 5.0 15.7 8.2
10.50 20.30 5.00 15.70 8.20
11.94
PS C:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septiana-Salsabila>
```

Penjelasan :

Program diatas dibuat menggunakan bahasa pemrograman Golang untuk mengolah data berat ikan yang akan dikelompokkan ke dalam wadah berdasarkan jumlah ikan dan kapasitas per wadah yang diinputkan. Program ini memanfaatkan tipe data array statis berkapasitas 1000 elemen bertipe float64 sebagai tempat penyimpanan data berat ikan. Pemrosesan data dilakukan melalui dua fungsi terpisah, yaitu hitungTotalWadah yang bertugas menjumlahkan berat ikan secara berurutan per kelompok sebanyak y elemen

menggunakan perulangan bersarang dengan increment manual, serta hitungRataRata yang menghitung nilai rata-rata dari total berat setiap wadah yang telah dihasilkan. Pada fungsi utama, program membaca jumlah ikan dan kapasitas wadah pada baris pertama, dilanjutkan dengan pembacaan berat masing-masing ikan, kemudian memanggil kedua fungsi tersebut untuk memproses data dan mencetak hasilnya ke layar dalam format dua angka desimal, sehingga alur eksekusi berjalan terstruktur, efisien, dan mampu menghasilkan keluaran berupa deretan total berat per wadah serta nilai rata-ratanya secara tepat.

3. Soal Unguided 3

   Pos Pelayanan Terpadu (posyandu) sebagai tempat pelayanan kesehatan perlu mencatat data berat balita (dalam kg). Petugas akan memasukkan data tersebut ke dalam array. Dari data yang diperoleh akan dicari berat balita terkecil, terbesar, dan reratanya.

   Buatlah program dengan spesifikasisubprogram sebagai berikut:

```
type arrBalita [100]float64

func hitungMinMax(arrBerat arrBalita; bMin, bMax *float64) {
/* I.S. Terdefinisi array dinamis arrBerat
   Proses: Menghitung berat minimum dan maksimum dalam array
   F.S. Menampilkan berat minimum dan maksimum balita */
     ...
}

function rerata (arrBerat arrBalita) real {
/* menghitung dan mengembalikan rerata berat balita dalam array */
     ...
}
```

   Perhatikan sesi interaksi pada contoh berikut ini (**teks bergaris bawah adalah input/read**)

```
Masukan banyak data berat balita : 4
Masukan berat balita ke-1: 5.3
Masukan berat balita ke-2: 6.2
Masukan berat balita ke-3: 4.1
Masukan berat balita ke-4: 9.9
Berat balita minimum: 4.10 kg
Berat balita maksimum: 9.90 kg
Rerata berat balita: 6.38 kg
```

Fakultas Informatika

Jawaban :

```
package main

import "fmt"

type arrBalita [100]float64

func hitungMinMax(arrBerat arrBalita, n int, bMin, bMax
*float64) {
    var j int = 1
    *bMin = arrBerat[0]
    *bMax = arrBerat[0]

    for j < n {
        if *bMin > arrBerat[j] {
            *bMin = arrBerat[j]
        }
        if *bMax < arrBerat[j] {
            *bMax = arrBerat[j]
        }
        j = j + 1
    }
}

func rerata(arrBerat arrBalita, n int) float64 {
    var sum float64 = 0
    var i int = 0

    for i < n {
        sum = sum + arrBerat[i]
        i = i + 1
    }
    return sum / float64(n)
}

func main() {
    var n int
    var berat arrBalita
    var min, max float64

    fmt.Print("Masukan banyak data berat balita : ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Printf("Masukan berat balita ke-%d : ", i+1)
        fmt.Scan(&berat[i])
    }

    hitungMinMax(berat, n, &min, &max)
    avg := rerata(berat, n)
```

```
    fmt.Printf("\nBerat balita minimum : %.2f kg\n", min)
    fmt.Printf("Berat balita maksimum : %.2f kg\n", max)
    fmt.Printf("Rerata berat balita : %.2f kg\n", avg)
}
```

Output :

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   Code

PS C:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septiana-Salsabila> go run "c:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septi
ana-Salsabila\Week 9 - Modul 10\Unguided\unguided-no3\no3.go"
Masukan banyak data berat balita : 4
Masukan berat balita ke-1 : 5.3
Masukan berat balita ke-2 : 6.2
Masukan berat balita ke-3 : 4.1
Masukan berat balita ke-4 : 9.9

Berat balita minimum : 4.10 kg
Berat balita maksimum : 9.90 kg
Rerata berat balita : 6.38 kg
PS C:\Users\amand\OneDrive\Dokumen\109082500145_Amanda-Septiana-Salsabila>
```

Penjelasan :

Program diatas dibuat menggunakan bahasa pemrograman Golang untuk mengimplementasikan algoritma pencarian nilai ekstrim dan perhitungan statistik dasar guna mencatat serta menganalisis data berat balita pada pos pelayanan terpadu, di mana program memanfaatkan array statis berkapasitas 100 elemen bertipe float64 sebagai tempat penyimpanan data. Logika pencarian dijalankan melalui fungsi hitungMinMax yang menerima parameter pointer float64 agar nilai minimum dan maksimum dapat langsung diperbarui pada variabel di fungsi pemanggil tanpa perlu return value, dengan mekanisme inisialisasi elemen pertama sebagai acuan kemudian dibandingkan secara sekuensial terhadap elemen berikutnya menggunakan perulangan yang diincrement secara manual. Sementara itu, fungsi rerata bertugas menjumlahkan seluruh elemen berat lalu membaginya dengan jumlah data untuk menghasilkan nilai rata-rata. Pada fungsi utama, program menangani input jumlah data dan berat masing-masing balita secara berurutan, memanggil kedua subprogram tersebut, serta mencetak hasil keluaran dalam format dua angka desimal berserta satuan kilogram, sehingga alur eksekusi berjalan terstruktur, efisien, dan mampu menghasilkan keluaran berupa nilai minimum, maksimum, serta rata-rata berat balita secara tepat.

D. Kesimpulan

Berdasarkan praktikum yang dilaksanakan pada hari Kamis, dapat disimpulkan bahwa algoritma pencarian nilai maksimum dan minimum bekerja secara sekuensial dengan menetapkan elemen pertama sebagai acuan, kemudian

membandingkannya secara iteratif hingga seluruh data terverifikasi untuk mendapatkan nilai ekstrim yang valid. Implementasi algoritma ini telah diterapkan pada array bertipe data dasar maupun terstruktur, serta memanfaatkan pointer dalam bahasa Go untuk memodifikasi beberapa variabel keluaran secara efisien tanpa bergantung pada return value ganda. Melalui latihan soal pendataan berat kelinci, pengelompokan berat ikan per wadah, dan statistik berat balita di posyandu, pendekatan pencarian max/min terbukti efektif dengan kompleksitas waktu O(n) untuk pengolahan data statistika dasar. Dengan penerapan inisialisasi variabel eksplisit, perulangan dengan increment manual, serta format output presisi dua desimal, pemahaman mengenai logika array dan implementasi algoritma pencarian dalam bahasa Go telah tercapai secara utuh.